

Specyfikacja Aplikacji Desktopowej – CSAimBot

1. Wymagania Funkcjonalne Aplikacji

a. Moduł Komunikacji

Moduł ten odpowiada za ustanowienie, utrzymanie i monitorowanie stabilnego, dwukierunkowego połączenia między komputerem PC a mikrokontrolerem zarządzającym sprzętem, a także za obsługę środowiska symulacyjnego:

- Nawiązywanie Połączenia
 - Wygląd (UI): Główny przycisk "Połącz / Rozłącz", inicjujący komunikację. Etykieta przycisku zmienia się dynamicznie w zależności od aktualnego stanu.
 - Zasada działania (Backend): Po wciśnięciu przycisku otwierana jest komunikacja PC-Mikrokontroler poprzez UART (USB). Następuje utworzenie połączenia między komputerem, a mikrokontrolerem. W tle realizowana jest procedura nawiązywania połączenia – aplikacja PC wysyła ramkę testową (np. INIT_CSBOT), a mikrokontroler musi odpowiedzieć z góry zdefiniowanym potwierdzeniem. Zabezpiecza to przed przypadkowym wysłaniem komend ruchu do innego urządzenia podpiętego pod USB.
- Status Połączenia i Monitorowanie Logów
 - Wygląd (UI): Wizualny wskaźnik (np. dioda LED w interfejsie: Zielona = połączono, Czerwona = błąd) informujący o poprawnym połączeniu lub błędzie.
 - Zasada działania (Backend): Obsługa komunikacji działa w osobnym wątku aplikacji (tzw. Thread/Worker), aby nasłuchiwanie na porcie nie blokowało interfejsu graficznego użytkownika.
- Przełącznik Trybu Pracy (Fizyczny / Symulacja)
 - Wygląd (UI): Przełącznik typu *Toggle* lub osobne zakładki pozwalające wybrać między pracą na fizycznym urządzeniu ("Tryb Sprzętowy") a symulacją ("Tryb ROS").
 - Zasada działania (Backend): Moduł obsługuje komunikację Windows -> ROS na dockerze. Po aktywacji tego trybu, wyliczone z systemu wizyjnego wektory ruchu nie są wysyłane na port UART, lecz przekierowywane (np. poprzez protokół TCP/UDP) do kontenera, co pozwala na symulację robota w Gazebo. Środowisko to wspiera również pełny obieg danych, włączając w to komunikację ROS->(Windows)->Robot, jeśli wymagana będzie integracja sprzętowa w pętli symulacji.

b. Moduł Kalibracji i Diagnostyki

Sekcja niezbędna do przygotowania robota do pracy z różnymi modelami myszy i ustawieniami systemowymi:

- Bazowanie (Homing):
 - Wygląd (UI): Przycisk "Bazuj robota (HOME)". Obok znajdują się kontrolki stanu osi X oraz Y (np. czerwone ikony zmieniające się na zielone po udanym bazowaniu) oraz tekstowe wyświetlanie aktualnych współrzędnych maszynowych.
 - Flow użytkownika: Po uruchomieniu urządzenia użytkownik musi kliknąć ten przycisk, zanim podejmie jakąkolwiek inną akcję. Opcjonalnie program może wymuszać tę akcję przy każdym nowym połączeniu z portem szeregowym.
 - Zasada działania (Backend): Po wciśnięciu przycisku aplikacja wysyła dedykowaną ramkę danych (np. komendę HOME) przez UART do mikrokontrolera. Mikrokontroler zaczyna podawać impulsy na sterowniki silników krokowych osi X i Y, wymuszając ruch wózka w kierunku punktu zerowego. Następnie wysyłany jest sygnał przerwania (interrupt) do mikrokontrolera, który natychmiast zatrzymuje silniki i przypisuje tej pozycji wartość $X=0$, $Y=0$. Od tego momentu aplikacja wie, gdzie znajduje się środek podkładki i jaki jest maksymalny bezpieczny zakres ruchu.
- Kalibracja DPI:
 - Wygląd (UI): Przycisk "Rozpocznij Kalibrację".
 - Flow użytkownika: Uruchamiana jest specjalna mapa w CS:GO z punktami referencyjnymi. Użytkownik ustawia mysz na środku obszaru roboczego i uruchamia proces.
 - Zasada działania (Backend): Algorytm musi ustalić relację między fizycznym ruchem myszy a ruchem celownika na ekranie. Robot będzie sprawdzał, o ile musi się poruszyć, żeby przejść między punktami referencyjnymi na specjalnej mapie. W tym celu przewidujemy wykorzystanie dokładnie trzech stałych punktów referencyjnych (np. tworzących trójkąt w przestrzeni gry). Aplikacja wysyła do robota rozkaz przesunięcia się w stronę każdego z tych punktów o ściśle określoną, znaną odległość (np. określoną liczbę kroków silnika). Robot wykonuje ruch, a następnie system wizyjny przechwytyjący ekran analizuje przesunięcie celownika względem tych trzech punktów. Wiedząc, o ile kroków obróciły się silniki w obu osiach podczas przejścia między poszczególnymi punktami i o ile pikseli przesunął się obraz, aplikacja oblicza bardzo precyzyjny współczynnik proporcjonalności. Dzięki temu podczas właściwej gry, gdy YOLO wykryje głowę przeciwnika w odległości 60 pikseli, aplikacja błyskawicznie przeliczy to na konkretną liczbę kroków do wysłania do sterowników silników.

- Mapa kalibracyjna:

Komenda w terminalu dla CSGO:

```
steam.exe -applaunch 730 +map nazwa_mapy -insecure
```

- -applaunch 730: Informuje Steama, że chcemy włączyć CS:GO.
- +map nazwa_mapy: Zmusza grę do natychmiastowego załadowania konkretnej mapy z folderu csgo/maps/ z pominięciem głównego menu.
- -insecure: Uruchamia grę bez ochrony VAC (Valve Anti-Cheat). Blokuje ona możliwość wejścia na serwery publiczne, eliminując ryzyko nałożenia blokady na konto użytkownika podczas testów.

Użycie wbudowanej biblioteki subprocess, która będzie podłączona z przyciskiem "Rozpocznij Kalibrację" w aplikacji.

Kod windows:

```
import subprocess
import os

def uruchom_kalibracje():
    steam_path = r"C:\Program Files (x86)\Steam\steam.exe"
    map_name = "aimbot_calibration_map"

    command = [
        steam_path,
        "-applaunch", "730",          # ID gry
        "+map", map_name,             # Załaduj mapę
        "-insecure",                  # Wyłącz VAC
        "-windowed", "-w", "1920", "-h", "1080" # Wymuś okno i rozdzielczość
    ]

    try:
        print("Uruchamianie CS:GO z mapą kalibracyjną...")
        subprocess.Popen(command)
    except Exception as e:
        print(f"Błąd podczas uruchamiania: {e}")
```

2. Kod w Pythonie (Wersja Linux)

```
import subprocess

def uruchom_kalibracje_linux():
    map_name = "aimbot_calibration_map"

    command = [
        "steam",
        "-applaunch", "730",          # ID gry
        "+map", map_name,             # Załaduj mapę
        "-insecure",                  # Wyłącz VAC
        "-windowed", "-w", "1920", "-h", "1080" # Wymuś okno i rozdzielczość
    ]

    try:
        print("Uruchamianie CS:GO na Linuxie z mapą kalibracyjną...")
        subprocess.Popen(command)
    except FileNotFoundError:
        print("Błąd: Nie znaleziono wykonywalnego pliku 'steam'.")
    except Exception as e:
        print(f"Błąd podczas uruchamiania: {e}")
```

- Test chwytaka i klikacza:
 - Wygląd (UI): Panel diagnostyczny zawierający przyciski "Zaciśnij / Zwolnij Chwytek" oraz przyciski wyzwalające lewy (LPM) i prawy (PPM) przycisk myszy. Warto dodać suwak do precyzyjnej regulacji siły docisku, aby nie zgnieść obudowy myszki.
 - Flow użytkownika: Użytkownik wykorzystuje ten panel do wstępnego ustawienia robota po umieszczeniu nowej myszki.
 - Zasada działania (Backend):
 - a) Chwytek: Aplikacja wysyła docelową wartość wychylenia. Mikrokontroler przekłada tę wartość na sygnał PWM (Pulse Width Modulation) o odpowiednim wypełnieniu, wysyłany do serwomechanizmu. Serwo obraca orczyk, zamykając wydrukowany z TPU chwytek, który utrzymuje mysz siłą tarcia.
 - b) Klikacz: Zgodnie ze specyfikacją, do naciskania klawiszy użyte będą solenoidy. Po kliknięciu "Test LPM" w aplikacji, program wysyła krótką komendę uderzenia. Mikrokontroler wysyła pojedynczy, bardzo krótki impuls prądowy na cewkę solenoidu. Pole magnetyczne błyskawicznie wypycha rdzeń z odpowiednią siłą, fizycznie wciskając przycisk myszy leżącej pod

robotem, po czym rdzeń wraca do pozycji spoczynkowej.

c. Moduł Bezpieczeństwa

Ten moduł odpowiada za ochronę sprzętu (robot, komputera i urządzeń peryferyjnych) przed uszkodzeniami mechanicznymi oraz za zachowanie kontroli użytkownika nad pracą urządzenia w sytuacjach awaryjnych.

- Awaryjne Zatrzymanie (E-Stop)
 - Wygląd (UI): Czerwony przycisk "E-STOP / ZATRZYMAJ". Dodatkowo, w ustawieniach aplikacji musi istnieć możliwość przypisania globalnego skrótu klawiszowego (np. Spacja lub Escape), który zadziała nawet wtedy, gdy aplikacja zminimalizowana jest w tle podczas gry w CS:GO.
 - Flow użytkownika: Gracz ma do dyspozycji przycisk zatrzymania robota. W przypadku nagłego, niekontrolowanego ruchu ramienia lub zablokowania mechanizmu, gracz uderza w zaprogramowany klawisz. Robot natychmiast zatrzymuje się w miejscu, a w aplikacji pojawia się komunikat o trybie awaryjnym.
 - Zasada działania (Backend): Aplikacja wymusza natychmiastowe wysłanie priorytetowej ramki danych (np. komendy HALT) przez interfejs UART. Mikrokontroler odbiera tę komendę jako przerwanie o najwyższym priorytecie. Logika sprzętowa natychmiast przestaje generować sygnały krokowe (STEP/DIR) dla sterowników silników. Dodatkowo, mikrokontroler wysyła sygnał dezaktywujący (np. pin ENABLE na sterownikach silników krokowych), co zdejmuje moment trzymający z cewek silników. Dzięki temu wózek osi X/Y traci sztywność i gracz może bezpiecznie, ręcznie odsunąć ramię robota od myszki bez ryzyka uszkodzenia napędów.
- Monitorowanie Integralności i Zgubionych Kroków - opcjonalne
 - Wygląd (UI): Panel wskaźników statusu (np. zielone/czerwone kropki) dla każdej osi z osobna (X, Y, Z). W razie awarii pojawia się pop-up z dokładnym kodem błędu (np. Błąd pozycjonowania osi X - zablokowany wózek). W panelu znajduje się również dostęp do zmiany parametrów robota.
 - Flow użytkownika: Jeśli podczas pracy robot nagle uderzy w przeszkodę i zgubi kroki, aplikacja automatycznie przerwie jego pracę, aby nie pogorszyć sytuacji. Nastawy robota będą mogły być zmienione w przypadku poluzowania się części lub wykrycia innych nieprawidłowości. Użytkownik musi wtedy zweryfikować stan mechaniczny, dokręcić elementy i przeprowadzić ponowne bazowanie.
 - Zasada działania (Backend): Algorytm opiera się na sprzężeniu zwrotnym z układu napędowego. Pozycja silników krokowych będzie odczytywana z enkoderów podłączonych w odpowiedni sposób do silników. Mikrokontroler nieustannie porównuje zadaną pozycję z faktyczną. Jeśli różnica między tymi wartościami przekroczy bezpieczny próg błędu, system wniośkuje, że wystąpiła fizyczna blokada

lub przeciążenie. Aplikacja natychmiast uruchamia procedurę E-Stop, ponieważ wszystkie komponenty muszą być sprawne, aby projekt działał poprawnie.

d. Moduł Statystyk i Monitorowania

Sekcja analityczna pełni rolę narzędzia diagnostycznego oraz walidacyjnego. Jej głównym zadaniem jest ciągle zbieranie danych telemetrycznych i wizyjnych w celu weryfikacji, czy fizyczny układ (robot + system wizyjny) spełnia rygorystyczne założenia projektowe dotyczące czasu reakcji i celności:

- Monitorowanie Czasu Reakcji

- Wygląd (UI): Dynamiczny wykres liniowy (odświeżany w czasie rzeczywistym) pokazujący opóźnienia w milisekundach. Obok wykresu znajdują się liczniki: "Aktualny czas reakcji", "Średni czas reakcji" oraz wskaźnik wizualny (np. zielony/czerwony), który zapala się na czerwono, gdy opóźnienie przekroczy założone w projekcie maksimum (<200 ms).
- Flow użytkownika: Użytkownik nie musi aktywnie ingerować w ten panel podczas gry. Po zakończeniu rundy lub meczu może wejść w tę zakładkę, aby sprawdzić czas reakcji.
- Zasada działania (Backend): Aplikacja precyzyjnie loguje znaczniki czasu (timestamps) w trzech kluczowych momentach trwania pętli sterowania:
 - a) T1 (Detekcja): Moment, w którym algorytm YOLO z sukcesem zidentyfikuje przeciwnika (bounding box) i zaklasyfikuje go jako priorytet (np. najbliżej celownika).
 - b) T2 (Transmisja): Moment wysłania przeliczonego wektora ruchu przez port UART do mikrokontrolera.
 - c) T3 (Akcja sprzętowa): Moment odebrania przez PC sprzętowego potwierdzenia od mikrokontrolera, że pierwsze impulsy zostały wysłane na sterowniki silników.Całkowity czas reakcji układu to różnica między T3 i T1. Aplikacja uśrednia te wyniki, by zniwelować błędy pomiarowe.

- Skuteczność i Celność

- Wygląd (UI): Panel w formie "Dashboardu" zawierający wskaźniki procentowe (np. wykresy). Podzielony na dwie główne kategorie: "Celność – Cele Statyczne" oraz "Celność – Cele Poruszające się". Widoczny jest też ogólny licznik oddanych strzałów do strzałów trafionych.
- Flow użytkownika: Gracz może przeglądać skuteczność w poprzednich rozgrywkach.

- Zasada działania (Backend): Jest to jeden z bardziej zaawansowanych mechanizmów aplikacji, opierający się na analizie klatek z gry:
 - a) Rozpoznawanie ruchu celu: System analizuje zmianę pozycji tzw. "bounding boxa" przeciwnika między kolejnymi klatkami obrazu. Jeśli środek ramki pozostaje w tym samym miejscu (z uwzględnieniem drobnego marginesu błędu), cel uznawany jest za statyczny. Jeśli ramka wyraźnie przesuwa się po osi X/Y, cel traktowany jest jako poruszający się.
 - b) Weryfikacja trafienia: Aplikacja zlicza każdą komendę aktywacji klikacza (solenoidu) wysłaną do robota. Następnie, system wizyjny analizuje np. pojawienie się czerwonego markera trafienia (w przypadku włączonych odpowiednich wskaźników w grze) lub analizuje killfeed w prawym górnym rogu ekranu, aby potwierdzić eliminację. Wynik jest zapisywany w odpowiedniej kategorii (cel ruchomy/statyczny).
- Zapis i Eksport Danych - opcjonalne
 - Wygląd (UI): Przycisk "Eksportuj raport sesji (CSV/JSON)" oraz tabela wyświetlająca historię zapisanych meczów.
 - Flow użytkownika: Użytkownik pobiera plik w celu analizy danych i wygenerowania wykresów.
 - Zasada działania (Backend): Program działający w tle zapisuje do lokalnej, lekkiej bazy danych lub pliku tekstowego kluczowe parametry użyte w danej sesji: użyte DPI, aktualny współczynnik kalibracji pikseli na kroki silnika, rodzaj używanej broni odczytanej przez YOLO oraz zebrane logi skuteczności i czasu reakcji.

2. Research Technologiczny

a. Wybór frameworka GUI: Porównanie Tkinter vs. PyQt

Projekt CSAimBot narzuca rygorystyczne wymagania architektoniczne. Aplikacja okienkowa musi w czasie rzeczywistym integrować trzy wymagające procesy: analizę obrazu przez model sztucznej inteligencji (YOLO), nieprzerwaną dwukierunkową komunikację sprzętową (UART z mikrokontrolerem) oraz dynamiczne renderowanie interfejsu użytkownika.

Ze względu na to, że rdzeń logiczny i wizyjny systemu zaimplementowano w języku Python, podjęto decyzję o realizacji interfejsu graficznego w tym samym środowisku, aby wyeliminować opóźnienia wynikające z komunikacji międzyprocesowej (IPC). W ramach researchu przeanalizowano i zestawiono ze sobą dwa główne podejścia: bibliotekę Tkinter (wraz z nowoczesnym forkiem CustomTkinter) oraz framework PyQt6.

- Biblioteka Tkinter

Tkinter to domyślna biblioteka graficzna wbudowana w standardową dystrybucję języka Python. Jej standardowy, surowy wygląd można unowocześnić za pomocą zewnętrznej biblioteki CustomTkinter, która wprowadza nowoczesne elementy interfejsu.

Zalety:

- Brak konieczności instalacji złożonych zależności: Tkinter działa "out-of-the-box" na każdej instalacji Pythona, co znacznie upraszcza proces dystrybucji i pakowania aplikacji.
- Niskie zapotrzebowanie na zasoby: Aplikacje napisane w tym środowisku uruchamiają się błyskawicznie i zużywają minimalne ilości pamięci RAM, co pozostawia więcej zasobów dla obciążającego modelu YOLO.

Ograniczenia:

- Brak natywnego bezpieczeństwa wątkowego (Not thread-safe): Jest to istotne wyzwanie z perspektywy naszego projektu. Biblioteka źle znosi modyfikacje interfejsu z poziomu innych wątków. Równoległe uruchomienie ciągłego nasłuchiwanie portu UART, pętli YOLO i interfejsu graficznego za pomocą standardowej biblioteki threading stwarza ryzyko "zamrażania" okna aplikacji.
- Wydajność multimedialna: Osadzanie dynamicznych, szybko odświeżających się strumieni wideo (klatki z OpenCV) oraz renderowanie płynnych wykresów w czasie rzeczywistym wymaga stosowania nieoptymalnych obejść i mocno obciąża główną pętlę programu.

- Framework PyQt6

PyQt6 to pythonowy port frameworka Qt, opierającego się na języku C++. Jest to rozwiązanie stosowane powszechnie w przemyśle do tworzenia zaawansowanego oprogramowania, w tym systemów sterowania maszynami.

Zalety:

- Rozbudowana architektura wielowątkowości: Framework posiada natywne, bardzo stabilne rozwiązanie problemu wielowątkowości (QThread). Wątki poboczne mogą bezkolizyjnie wymieniać dane z głównym wątkiem GUI za pomocą asynchronicznego mechanizmu Sygnałów i Slotów, co gwarantuje pełną responsywność interfejsu nawet pod dużym obciążeniem.

- Graficzne środowisko projektowe: Dostępność narzędzia Qt Designer pozwala na wizualne projektowanie układu okien i ich bezproblemowy eksport do kodu Pythona.
- Wysoka wydajność renderowania: Środowisko jest przystosowane do płynnej obsługi strumieni wideo oraz dynamicznych danych (np. za pomocą zintegrowanych bibliotek do rysowania wykresów).

Ograniczenia:

- Większy rozmiar i złożoność: Biblioteka zajmuje znacznie więcej pamięci niż Tkinter, co przełoży się na większy rozmiar ostatecznego pliku instalacyjnego. Wymaga również pobrania dodatkowych, zewnętrznych paczek przez menedżer pip.
- Wysoki próg wejścia: Zrozumienie specyficznej dla Qt pętli zdarzeń oraz paradygmatu sygnałów i slotów wymaga poświęcenia większej ilości czasu na etapie wdrożenia.

b. Odpalanie CS:GO na Linuxie i system wizyjny

Uruchomienie gry CS:GO na systemie Linux oraz bezpośrednia integracja aplikacji analizującej obraz wymaga uwzględnienia fundamentalnych różnic w architekturze wyświetlania obrazu w porównaniu do systemu Windows.

Środowisko uruchomieniowe gry na Platformie Steam udostępnia w pełni natywnego klienta gry na systemy z rodziny Linux. Oznacza to, że silnik gry komunikuje się bezpośrednio ze sterownikami karty graficznej (wykorzystując API Vulkan lub OpenGL), bez pośrednictwa jakichkolwiek warstw kompatybilności. Z punktu widzenia naszego projektu jest to sytuacja optymalna – natywne wykonanie minimalizuje narzut procesora i pozwala na stabilne utrzymanie założonych w projekcie minimum 60 FPS. Dodatkowo, aby zrealizować wymagania sprzętowe, komputer musi posiadać zainstalowane własnościowe sterowniki (np. NVIDIA) zapewniające obsługę rdzeni CUDA, które są niezbędne dla szybkiej inferencji modelu YOLO.

Jako narzędzie do pobierania klatek wybrano bibliotekę bettercam. O ile na systemie Windows szybkie przechwytywanie pamięci z karty graficznej (np. poprzez DXGI Desktop Duplication) jest natywne i zunifikowane, o tyle na środowisku Linux architektura ta jest znacznie bardziej pofragmentowana i zależy od używanego serwera wyświetlania:

- Środowisko X11 (X.Org Server): Jest to tradycyjny menedżer okien. Z inżynierskiego punktu widzenia X11 pozwala na stosunkowo łatwy dostęp do głównego bufora ramki (root window). Biblioteki pythonowe do zrzutów ekranu mogą w tym środowisku wykorzystywać rozszerzenie XShm (MIT Shared Memory Extension). Pozwala ono na ominięcie kopiowania danych przez sieć serwera X i bezpośredni odczyt pikseli z pamięci współdzielonej. Gwarantuje to niezwykle niskie opóźnienia, co jest absolutnie kluczowe dla dostarczenia klatek do modelu YOLO w czasie rzeczywistym.

- Środowisko Wayland: Jest to nowoczesny protokół, który został zaprojektowany z myślą o ścisłym bezpieczeństwie i izolacji procesów. Architektura Waylanda domyślnie blokuje aplikacjom trzecim możliwość bezpośredniego czytania pikseli z okien innych aplikacji (czyli CS:GO). Aby przechwycić obraz na Waylandzie, nasza aplikacja musiałaby komunikować się przez złożony system PipeWire oraz interfejsy D-Bus (xdg-desktop-portal). Wprowadza to ogromny narzut technologiczny, wymusza kompresję/dekompresję strumienia i znacząco podnosi opóźnienia.

Głównym założeniem projektu jest utrzymanie całkowitego czasu reakcji robota poniżej 200 ms. Biorąc pod uwagę czas potrzebny na inferencję sieci YOLO oraz transmisję danych przez UART do mikrokontrolera, budżet czasowy na samo przechwycenie klatki jest skrajnie mały.

W związku z tym, wymogiem koniecznym do poprawnego działania aplikacji na systemie Linux jest uruchomienie sesji opartej wyłącznie na serwerze X11. Środowisko Wayland, ze względu na swoje restrykcje bezpieczeństwa i opóźnienia w przechwytywaniu strumienia, całkowicie dyskwalifikuje się w zastosowaniach wymagających milisekundowej precyzji w analizie klatek z gier.

3. Tworzenie instalki (.exe i Linux)

W celu udostępnienia aplikacji użytkownikom końcowym, konieczne jest przekształcenie kodu źródłowego w samodzielne pakiety instalacyjne. Do utworzenia pakietów instalacyjnych zostaną użyte dedykowane narzędzia dla poszczególnych platform.

a. Środowisko Windows

Dystrybucja na systemy Windows zostanie podzielona na dwa etapy:

- Budowanie pliku wykonywalnego (.exe): Zastosowane zostanie narzędzie PyInstaller. Skrypt kompilujący użyje flag wymuszających tryb okienkowy (bez konsoli w tle) oraz architekturę katalogową, co jest zalecane przy dużych projektach AI.

Przykładowa komenda w terminalu (Windows):

```
pyinstaller --noconsole --onedir --name "CSAimBot" --add-data
"models/yolo_weights.pt;models/" main.py
```

(Flaga `--add-data` gwarantuje, że model sztucznej inteligencji zostanie prawidłowo dołączony do folderu wyjściowego `dist/CSAimBot`).

- Tworzenie kreatora instalacji: Do utworzenia ostatecznego pakietu instalacyjnego użyte zostanie oprogramowanie Inno Setup. Skrypt pobierze wygenerowane przez PyInstaller pliki binarne i skompresuje je do pliku `setup.exe`, tworząc niezbędne skróty.

Przykładowy fragment skryptu konfiguracyjnego (`CSAimBot_Installer.iss`):

```
[Setup]
AppName=CSAimBot
AppVersion=1.0
DefaultDirName={autopf}\CSAimBot
OutputDir=.\\Installer
OutputBaseFilename=CSAimBot_Setup
```

```
[Files]
; Pobranie całego wygenerowanego folderu z aplikacją
Source: "dist\CSAimBot\*"; DestDir: "{app}"; Flags: ignoreversion recursesubdirs
createallsubdirs
```

```
[Icons]
; Utworzenie skrótów na pulpicie i w menu Start
Name: "{autodesktop}\CSAimBot"; Filename: "{app}\CSAimBot.exe"
Name: "{group}\CSAimBot"; Filename: "{app}\CSAimBot.exe"
```

b. Środowisko Linux

Tworzenie pakietu na systemy z rodziny Linux wymaga innego podejścia, gwarantującego niezależność od dystrybucji.

- Kompilacja binarki natywnej: Podobnie jak na platformie Windows, kod źródłowy zostanie spakowany przy użyciu narzędzia PyInstaller uruchomionego natywnie w środowisku Linux (np. na systemie Ubuntu).

Przykładowa komenda w terminalu (Linux):

```
pyinstaller --noconsole --onedir --name "CSAimBot" --add-data
"models/yolo_weights.pt:models/" main.py
```

(Uwaga: w systemie Linux separator ścieżek w --add-data to dwukropek :, w przeciwieństwie do średnika ; w Windowsie).

- Tworzenie pakietu dystrybucyjnego (AppImage): Do utworzenia uniwersalnego pakietu instalacyjnego użyte zostanie narzędzie appimagetool. Wygenerowany wcześniej folder dist/CSAimBot zostanie zintegrowany ze specjalnym plikiem AppRun, który zarządza ścieżkami bibliotek, a następnie spakowany do jednego pliku wykonywalnego.

Przykładowy skrypt powłoki (build_appimage.sh) do automatyzacji procesu:

```
#!/bin/bash

echo "Kompilacja za pomocą PyInstaller..."
pyinstaller --noconsole --onedir --name "CSAimBot" --add-data
"models/yolo_weights.pt:models/" main.py
```

```
echo "Przygotowanie struktury AppDir..."
```

```
mkdir -p AppDir/usr/bin
```

```
cp -r dist/CSAimBot/* AppDir/usr/bin/
```

```
echo "Tworzenie pliku .desktop..."
```

```
cat > AppDir/CSAimBot.desktop << 'EOF'
```

```
[Desktop Entry]
```

```
Name=CSAimBot
```

```
Exec=CSAimBot
```

```
Icon=csaimbot
```

```
Type=Application
```

```
Categories=Utility;
```

```
EOF
```

```
echo "Generowanie AppImage..."
```

```
chmod +x appimagetool-x86_64.AppImage
```

```
./appimagetool-x86_64.AppImage AppDir CSAimBot-x86_64.AppImage
```

```
echo "Zakończono! Pakiet AppImage jest gotowy."
```